

GPU Optimizer Triton - Guia para el equipo

1. Idea principal del proyecto

Este proyecto busca responder una pregunta:

Como podemos hacer que un modelo de lenguaje genere kernels GPU mas correctos usando una gramatica formal?

Un LLM puede escribir codigo, pero no siempre lo hace bien. Cuando genera codigo GPU puede cometer errores como:

- sintaxis invalida,
- uso incorrecto de APIs de Triton,
- errores de memoria,
- codigo que compila pero calcula mal,
- codigo funcional pero ineficiente.

La idea del proyecto no es confiar ciegamente en el modelo. La idea es guiarlo con reglas.

El sistema debe comparar dos formas de generar kernels:

1. Generacion libre: el modelo genera codigo sin restricciones.
2. Generacion restringida: el modelo genera codigo siguiendo una gramatica formal.

Luego se evalua si la gramatica ayuda a generar codigo mas valido y correcto.

2. Que es un kernel GPU?

Un kernel GPU es una funcion que se ejecuta en paralelo sobre muchos datos.

Ejemplo simple:

$$C = A + B$$

En CPU, esto puede verse como una operacion normal. En GPU, queremos que

muchos elementos se sumen al mismo tiempo:

$C[0] = A[0] + B[0]$

$C[1] = A[1] + B[1]$

$C[2] = A[2] + B[2]$

...

Triton permite escribir kernels GPU en Python. Un kernel de suma de vectores normalmente tiene:

- imports de Triton,
- decorador '@triton.jit',
- una funcion kernel,
- calculo de offsets,
- mascara para no salir del arreglo,
- 'tl.load' para leer datos,
- 'tl.store' para escribir resultados.

3. Que es un LLM en este proyecto?

El LLM es el modelo que intenta escribir el kernel.

Ejemplo de prompt:

Generate a Triton kernel for vector addition.

The kernel should compute $Z = X + Y$ for N elements.

El modelo puede responder con codigo. El problema es que puede responder bien o mal.

Por eso el proyecto no solo pregunta:

Puede un modelo generar codigo?

Pregunta algo mas especifico:

Genera mejor codigo cuando lo restringimos con una gramatica?

4. Que es una gramatica formal?

Una gramatica formal es un conjunto de reglas que define que textos son validos.

En este proyecto, la gramatica debe describir una forma valida de kernel Triton.

Por ejemplo, una gramatica puede obligar a que el codigo tenga:

- 'import triton',
- 'import triton.language as tl',
- '@triton.jit',
- una funcion con argumentos esperados,
- 'tl.load',
- 'tl.store',
- una mascara 'offsets < N'.

La gramatica no decide si el codigo es rapido. Su primera funcion es limitar la forma del codigo para evitar salidas invalidas.

5. Que es constrained decoding?

Normalmente, un LLM genera texto token por token.

Sin restricciones, el modelo puede elegir cualquier token probable.

Con constrained decoding, la gramatica filtra los tokens permitidos en cada paso.

En simple:

Modelo quiere escribir codigo

|

Gramatica revisa que tokens son validos

|

Solo se permite elegir tokens que siguen las reglas

Esto es mas fuerte que un prompt.

Un prompt solo pide:

Por favor genera codigo valido.

La gramatica impone:

Solo puedes generar codigo con esta estructura.

6. Flujo esperado del sistema final

El flujo ideal del proyecto es:

Prompt de tarea

|

v

LLM genera kernel

|

+-- modo baseline: sin gramatica

|

+-- modo constrained: con gramatica

|

v

Codigo Triton generado

|

v

Validacion

|

+-- compila?

+-- corre?

+-- da el resultado correcto?

|

v

Comparacion de resultados

La comparacion importante es:

LLM sin gramatica vs LLM con gramatica

7. Que tenemos ya en el repo?

El repositorio ya tiene una base para trabajar:

- 'main.py': pipeline inicial del prototipo.
- 'models/classifier.py': clasifica snippets como element-wise, reduction o matrix operation.
- 'parsing/ast_parser.py': convierte codigo a AST.
- 'parsing/xgrammar_converter.py': crea una representacion estructurada tipo XGrammar.
- 'generation/kernel_templates.py': contiene templates de kernels Triton.
- 'generation/triton_generator.py': selecciona templates segun el tipo de problema.
- 'generation/constrained_decoder.py': primer prototipo de capa de restricciones; valida si un kernel generado cumple reglas basicas.
- 'parsing/triton_grammar_rules.py': reglas simples para revisar estructura Triton obligatoria.
- 'benchmarks/': scripts para medir tiempos y generar graficas.
- 'evaluation/model_evaluation.py': runner para comparar modelos en modo baseline y constrained.
- 'evaluation/collect_real_outputs.py': recolecta outputs reales desde Ollama, OpenAI y Anthropic.
- 'evaluation/real_outputs.jsonl': outputs reales ya recolectados para el segundo video.
- 'evaluation/experiment_log.md': bitacora del experimento de modelos.
- 'prompts/kernel_generation_prompt.txt': prompt base para pedir kernels.
- 'config/model_eval.yaml': configuracion de modelos para la evaluacion.
- 'data/kernel_generation_tasks.json': tareas de generacion, por ahora vector addition.
- 'tests/': pruebas automaticas.
- 'docs/': documentacion y planes de presentacion.

Importante:

La carpeta actual ya tiene una base de evaluacion con modelos reales y un prototipo de restricciones. Todavia no es el sistema final porque falta constrained decoding real token por token con XGrammar.

8. Como se conecta la explicacion con el codigo?

Esta seccion aterriza la idea en archivos reales del repositorio.

Flujo 1: pipeline inicial del prototipo

Cuando corren:

```
python main.py --code "C = A + B"
```

el flujo real es:

```
main.py
|
+-- models/classifier.py
|   decide si el codigo es element_wise, reduction, matrix_operation o generic
|
+-- parsing/ast_parser.py
|   convierte el snippet a una estructura AST en forma de diccionario
|
+-- parsing/xgrammar_converter.py
|   convierte esa AST a texto estructurado tipo XGrammar
|
+-- generation/triton_generator.py
|   escoge un template de kernel
|
+-- generation/kernel_templates.py
|   contiene el codigo Triton base
```

Ejemplo:

Entrada:

C = A + B

Salida:

Problem type: element_wise

Generated Triton kernel: elementwise_kernel

Este flujo sirve como prototipo y referencia. No es todavía constrained decoding real.

Flujo 2: evaluacion para el video con datos mock

Cuando corren:

```
python evaluation/model_evaluation.py --samples 3
```

el flujo real es:

```
evaluation/model_evaluation.py
|
+-- lee config/model_eval.yaml
|   modelos, tiers, provider, modos baseline/constrained
|
+-- lee data/kernel_generation_tasks.json
|   tarea actual: vector_add
|
+-- lee prompts/kernel_generation_prompt.txt
|   prompt base para todos los modelos
|
+-- genera outputs
|   hoy puede usar provider mock
|
+-- evalua checks simples
|   sintaxis valida?
|   tiene @triton.jit?
|   tiene tl.load y tl.store?
|   parece hacer x + y?
|
+-- escribe resultados
    evaluation/artifacts/model_eval_results.csv
    evaluation/artifacts/generated_kernels.jsonl
    evaluation/artifacts/model_eval_summary.md
```

Este flujo ayuda para el segundo video. Sirve para practicar la comparacion small model vs frontier models, pero los resultados mock no se deben presentar como resultados reales.

Flujo 2B: evaluacion para el video con outputs reales

Dilan agrego un flujo para recolectar outputs reales de modelos:

```
python evaluation/collect_real_outputs.py --provider ollama
python evaluation/collect_real_outputs.py --provider openai
python evaluation/collect_real_outputs.py --provider anthropic
```

Esto usa:

```
config/model_eval.yaml  
prompts/kernel_generation_prompt.txt  
data/kernel_generation_tasks.json
```

y guarda las respuestas en:

```
evaluation/real_outputs.jsonl
```

Despues se evaluan esos outputs reales con:

```
python evaluation/model_evaluation.py --manual-outputs evaluation/real_outputs.jsonl
```

Esto genera:

```
evaluation/artifacts/model_eval_results.csv  
evaluation/artifacts/generated_kernels.jsonl  
evaluation/artifacts/model_eval_summary.md
```

Resultados actuales del experimento:

```
Qwen2.5-Coder-1.5B baseline:  syntax 100%, kernel shape 33%, correctness proxy 0%  
Qwen2.5-Coder-1.5B constrained: syntax 100%, kernel shape 100%, correctness proxy 0%  
GPT-4o baseline/constrained:  syntax 100%, kernel shape 100%, correctness proxy 100%  
Claude baseline/constrained:  syntax 100%, kernel shape 100%, correctness proxy 100%
```

Interpretacion:

- Los modelos frontier generan kernels correctos para esta tarea simple.
- El modelo pequeno mejora en estructura cuando se le agrega el contrato constrained.
- El modelo pequeno todavia falla la metrica de correctness proxy.

- Esto sirve para el segundo video porque muestra una decision real: las restricciones ayudan en estructura, pero no sustituyen validacion funcional ni constrained decoding real.

Flujo 2C: prototipo de restricciones

Los cambios nuevos de main agregaron:

```
generation/constrained_decoder.py
parsing/triton_grammar_rules.py
tests/test_constrained_decoder.py
```

El flujo actual es:

```
codigo generado
|
v
generation/constrained_decoder.py
|
v
parsing/triton_grammar_rules.py
|
+-- revisa @triton.jit
+-- revisa def kernel(...)
+-- revisa tl.program_id(...)
+-- revisa tl.load(...)
+-- revisa tl.store(...)
+-- revisa parentesis y brackets balanceados
|
v
aceptado o rechazado
```

Este componente funciona como validador/post-check. Todavia no controla al modelo mientras genera tokens. Por eso se debe explicar como prototipo hacia constrained decoding, no como XGrammar final.

Flujo 3: benchmarks simples

Cuando corren:

```
python benchmarks/run_benchmarks.py --runs 5
```

el flujo es:

```
benchmarks/run_benchmarks.py
|
+-- lee data/sample_problems.json
|
+-- llama main.run_pipeline(code)
|
+-- mide tiempo de generacion
|
+-- guarda benchmarks/results.csv
```

Luego:

```
python benchmarks/plot_results.py
```

genera:

```
benchmarks/results_summary.png
```

Esto mide tiempo de generacion del prototipo, no rendimiento real del kernel en GPU.

9. Mapa de carpetas explicado

‘prompts/’

Aqui viven los prompts que se le dan al modelo.

Archivo principal:

```
prompts/kernel_generation_prompt.txt
```

Este prompt dice:

You are generating a Triton GPU kernel.

Task:

{task_prompt}

Return only Python code.

El programa reemplaza {task_prompt} con una tarea concreta, por ejemplo:

Generate a Triton kernel for vector addition.

En la practica, esta carpeta la usa la persona encargada de generacion con LLM.

'config/'

Aqui viven configuraciones que no deberian estar hardcodeadas en el codigo.

Archivo importante:

config/model_eval.yaml

Ese archivo define:

- modelos a comparar,
- si son small o frontier,
- provider,
- numero de muestras,
- modos 'baseline' y 'constrained'.

Ejemplo conceptual:

models:

- id: small_qwen25_coder_1_5b
- tier: small

provider: ollama
model_name: qwen2.5-coder:1.5b

Los providers actuales son:

- 'ollama': modelo pequeno local.
- 'openai': GPT-4o.
- 'anthropic': Claude Haiku 4.5.

Para OpenAI y Anthropic se necesitan API keys en variables de entorno o en .env.

'data/'

Aqui viven entradas del experimento.

Archivo importante:

data/kernel_generation_tasks.json

Define la tarea que el modelo debe resolver.

Ejemplo:

```
{  
  "id": "vector_add",  
  "operation": "vector addition",  
  "prompt": "Generate a Triton kernel for vector addition."  
}
```

Si despues agregan otra tarea, por ejemplo multiplicacion elemento a elemento, se agregaria aqui.

'evaluation/'

Aqui vive la comparacion de modelos.

Archivo importante:

evaluation/model_evaluation.py

Este script:

- carga modelos desde 'config/model_eval.yaml',
- carga tareas desde 'data/kernel_generation_tasks.json',
- arma prompts desde 'prompts/kernel_generation_prompt.txt',
- genera outputs,
- evalua checks simples,
- guarda resultados.

Tambien hay:

evaluation/collect_real_outputs.py

evaluation/real_outputs.jsonl

evaluation/experiment_log.md

collect_real_outputs.py llama proveedores reales y guarda respuestas.

real_outputs.jsonl contiene los outputs ya recolectados.

experiment_log.md resume decisiones, resultados y observaciones para el segundo video.

En la practica, esta carpeta es el punto central para el segundo video.

'generation/'

Aqui viven los templates de kernels Triton.

Archivos:

generation/kernel_templates.py

generation/triton_generator.py

generation/constrained_decoder.py

kernel_templates.py contiene codigo base de kernels, por ejemplo

elementwise_kernel.

triton_generator.py decide que template devolver segun el tipo de problema.

Esta carpeta no reemplaza al LLM. Sirve como referencia para saber como deberia verse un kernel valido.

constrained_decoder.py es un primer control layer. Hoy valida outputs ya generados. En el futuro debe conectarse a XGrammar para restringir tokens durante la generacion.

‘parsing/’

Aqui vive el analisis de codigo.

Archivos:

parsing/ast_parser.py
parsing/xgrammar_converter.py
parsing/dependency_graph.py
parsing/triton_grammar_rules.py

ast_parser.py convierte codigo Python a AST.

xgrammar_converter.py convierte la AST a texto estructurado. Ojo: esto no es constrained decoding real.

dependency_graph.py todavia es una base simple para representar dependencias.

triton_grammar_rules.py contiene reglas practicas para validar kernels
Triton: decorador, funcion, tl.program_id, tl.load, tl.store y simbolos balanceados.

‘benchmarks/’

Aqui viven mediciones simples.

Archivos:

benchmarks/run_benchmarks.py
benchmarks/plot_results.py
benchmarks/profiler.py

run_benchmarks.py mide tiempo de generacion.

plot_results.py genera una grafica.

profiler.py intenta detectar GPU/Triton y correr una prueba minima si el entorno lo permite.

'tests/'

Aqui viven pruebas automaticas.

Sirven para que cuando alguien cambie codigo, pueda revisar que no rompio lo basico:

python -m pytest -q

'docs/'

Aqui vive documentacion:

- guia del equipo,
- plan del segundo video,
- outline de presentacion.

10. Que archivo toca cada integrante?

Esta es una guia practica. No es una regla absoluta, pero ayuda a no pisarse.

Integrante 1: gramatica formal

Deberia crear o trabajar en:

grammars/

Ejemplos de archivos futuros:

grammars/vector_add.ebnf
grammars/vector_add_xgrammar.py
docs/grammar_notes.md

Su objetivo practico:

- escribir reglas para un kernel 'vector_add',
- definir que codigo es valido,
- listar ejemplos validos e invalidos.

Ejemplo de lo que podria definir:

Un kernel valido debe tener:

- import triton
- import triton.language as tl
- @triton.jit
- def vector_add_kernel(...)
- tl.load para X
- tl.load para Y
- tl.store para Z

Integrante 2: generacion con LLM

Deberia trabajar en:

prompts/
config/model_eval.yaml
evaluation/

Su objetivo practico:

- elegir modelos reales,
- correr el mismo prompt en todos,
- guardar outputs,

- documentar errores.

Archivos que probablemente toque:

prompts/kernel_generation_prompt.txt
config/model_eval.yaml
evaluation/model_evaluation.py

Integrante 3: constrained decoding

Deberia crear o trabajar en:

constrained/

Ejemplos de archivos futuros:

constrained/xgrammar_runner.py
constrained/constrained_generator.py

Su objetivo practico:

- cargar la gramatica,
- conectarla al modelo,
- generar texto restringido,
- devolver codigo generado.

Aqui se conectan las partes del Integrante 1 y 2.

Integrante 4: kernels ejecutables

Deberia trabajar en:

benchmarks/
generation/
tests/

Y probablemente crear:

validation/

Ejemplos de archivos futuros:

validation/compile_check.py
validation/run_triton_kernel.py
validation/correctness.py

Su objetivo practico:

- tomar un kernel generado,
- compilarlo,
- correrlo con tensores de prueba,
- comparar contra PyTorch.

11. Como se veria el sistema final en codigo?

Una version final simple podria verse asi:

```
data/kernel_generation_tasks.json
|
v
prompts/kernel_generation_prompt.txt
|
v
models reales / APIs / local model
|
+-- baseline_generator.py
|   genera codigo libre
|
+-- constrained_generator.py
|   genera codigo con gramatica
|
v
validation/correctness.py
    compila y prueba
```

|
v

evaluation/model_evaluation.py
guarda CSV, resumen y metricas

En palabras:

1. data/ define que tarea resolver.
2. prompts/ define como se le pide al modelo.
3. grammars/ define que codigo esta permitido.
4. constrained/ aplica la gramatica durante generacion.
5. validation/ revisa si el codigo funciona.
6. evaluation/ junta los resultados.

12. Ejemplo practico de una corrida

Supongamos que la tarea es vector addition.

La tarea vive en:

data/kernel_generation_tasks.json

El prompt base vive en:

prompts/kernel_generation_prompt.txt

El runner arma un prompt final:

You are generating a Triton GPU kernel.

Task:

Generate a Triton kernel for vector addition.

Return only Python code.

Luego corre dos modos:

baseline:

el modelo genera libremente

constrained:

el modelo genera usando la gramatica

Cada output se guarda en:

evaluation/artifacts/generated_kernels.jsonl

Cada resultado resumido se guarda en:

evaluation/artifacts/model_eval_results.csv

El CSV puede tener columnas como:

model_id, mode, syntax_valid, kernel_shape_valid, correctness_proxy

Esto permite comparar:

modelo pequeno baseline vs modelo pequeno constrained

frontier baseline vs frontier constrained

13. Que falta implementar en codigo?

Faltan tres bloques principales.

Bloque A: gramatica real

Crear una carpeta:

grammars/

Y definir una gramatica para vector_add.

Meta:

Que la gramatica permita generar solo una forma valida de kernel Triton para suma de vectores.

Bloque B: constrained decoding real

Crear una carpeta:

constrained/

Y conectar:

modelo + tokenizer + gramatica

Meta:

Que el modelo no solo reciba instrucciones, sino que realmente este limitado por la gramatica.

Bloque C: validacion real de kernels

Crear una carpeta:

validation/

Y probar:

- compila?
- corre?
- produce el mismo resultado que PyTorch?

Meta:

Convertir outputs de texto en evidencia real de código GPU funcional.

14. Que parte es solo prototipo?

Hay piezas que ayudan, pero no son todavía el requisito central:

- El 'xgrammar_converter.py' actual no aplica constrained decoding real. Solo genera una representación textual estructurada.
- 'llm_decider.py' tiene heurísticas, no un LLM real.
- Los resultados de 'evaluation/artifacts/' pueden ser mock si se generan con 'provider: mock'.
- Los templates de Triton son útiles como referencia, pero no equivalen a generación real por LLM.

Esto no está mal. Solo hay que explicarlo correctamente.

15. Que pide el reto?

El reto pide como mínimo:

1. Usar XGrammar o una gramática formal para constrained decoding.
2. Integrar la gramática con un LLM.
3. Generar código GPU funcional.
4. Comparar contra generación sin restricciones.
5. Medir al menos:
 - tasa de compilación exitosa,
 - corrección funcional,
 - una métrica adicional.
6. Documentar el trabajo y defenderlo.

Para el experimento final, lo mínimo razonable sería:

- una operación simple, por ejemplo vector addition,
- un modelo pequeño,
- dos modelos frontier como baseline para el video,
- varias generaciones sin gramática,
- varias generaciones con gramática,
- tabla comparativa de resultados.

16. División del trabajo entre 4 integrantes

La division recomendada es por partes tecnicas del sistema.

Integrante 1: Gramatica formal

Hace:

- disenar las reglas de kernels validos,
- definir que estructura de Triton se permite,
- decidir que errores debe bloquear la gramatica,
- mantener una version simple y usable.

Implementa:

- archivos de gramatica,
- ejemplos validos,
- ejemplos invalidos,
- documentacion tecnica de las reglas.

Explica en la defensa:

Como definimos que codigo es valido y que errores evita nuestra gramatica.

Integrante 2: Generacion con LLM

Hace:

- elegir modelos,
- preparar prompts,
- correr baseline sin restricciones,
- guardar outputs generados,
- mantener condiciones justas entre modelos.

Implementa:

- scripts o wrappers de generacion,
- prompts versionados,
- guardado de respuestas,
- configuracion de modelos.

Explica en la defensa:

Como el modelo recibe la tarea y como genera kernels GPU.

Integrante 3: Constrained decoding

Hace:

- conectar la gramatica con el modelo,
- aplicar restricciones durante la generacion,
- resolver problemas entre tokenizer, modelo y gramatica,
- producir outputs restringidos.

Implementa:

- integracion con XGrammar o herramienta equivalente,
- modo constrained,
- manejo de errores,
- comparacion basica contra modo libre.

Explica en la defensa:

Como logramos que la gramatica controle la generacion token por token.

Integrante 4: Kernels ejecutables

Hace:

- tomar codigo generado,
- preparar wrappers de ejecucion,
- compilar kernels,
- crear inputs de prueba,
- comparar contra PyTorch.

Implementa:

- validacion de compilacion,
- ejecucion en GPU o Colab,
- pruebas funcionales,
- casos de prueba reproducibles.

Explica en la defensa:

Como sabemos que el codigo generado no solo se ve bien, sino que corre y calcula correctamente.

17. Trabajo compartido

Aunque cada persona tenga una parte tecnica, hay cosas que deben hacer entre todos:

- definir el alcance final,
- decidir la operacion principal,
- revisar resultados,
- escribir el paper,
- preparar el video,
- preparar la defensa oral,
- asegurarse de que todos entienden el flujo completo.

Nadie debe quedar solo con paper o slides. Todos deben poder explicar una parte tecnica.

18. Como usar el repo

Instalacion:

```
pip install -r requirements.txt
```

Ejecutar el pipeline inicial:

```
python main.py --code "C = A + B"
```

Ejecutar benchmarks simples:

```
python benchmarks/run_benchmarks.py --runs 5  
python benchmarks/plot_results.py
```

Ejecutar evaluacion mock de modelos:

```
python evaluation/model_evaluation.py --samples 3
```

Correr pruebas:

```
python -m pytest -q
```

19. Que significa el prompt actual?

El archivo prompts/kernel_generation_prompt.txt contiene el prompt base:

You are generating a Triton GPU kernel.

Task:

{task_prompt}

Return only Python code. Do not include explanations.

La parte {task_prompt} se reemplaza por una tarea concreta.

Por ejemplo:

Generate a Triton kernel for vector addition.

The kernel should compute $Z = X + Y$ for N elements.

Ese prompt sirve para que todos los modelos reciban la misma instruccion.

Importante:

- El prompt no es la gramatica.
- El prompt pide comportamiento.
- La gramatica restringe formalmente la salida.
- Constrained decoding aplica la gramatica durante la generacion.

20. Que deben evitar decir?

No conviene decir:

Nuestro sistema ya optimiza cualquier kernel GPU automaticamente.

Tampoco:

Ya usamos XGrammar real para restringir tokens.

Si todavia no esta conectado, hay que decirlo como trabajo en progreso.

Una forma honesta de decirlo:

Actualmente tenemos una infraestructura inicial para generar, evaluar y comparar kernels. El siguiente paso central es conectar una gramatica formal con el LLM para constrained decoding real.

21. Que deberia estar listo para el segundo video?

Para el video del 23 de mayo, el foco no es explicar todo el pipeline. El foco es contar decisiones y aprendizajes sobre modelos.

Deben tener:

- 1 modelo pequeno,
- 2 modelos frontier,
- mismo prompt para todos,
- algunos outputs generados,
- errores observados,
- comparacion honesta,
- que aprendieron y que van a cambiar.

El video debe explicar:

- por que eligieron esos modelos,
- que trade-offs hay,
- que proveedor usaron,
- que se mantuvo constante,
- que errores salieron,
- que sigue.

22. Roadmap recomendado

Semana actual

- Elegir los tres modelos para el video.

- Correr el prompt base en los tres modelos.
- Guardar outputs reales.
- Preparar narrativa del video.

Siguiente etapa

- Crear gramatica formal para vector addition.
- Probar constrained decoding real.
- Comparar outputs con y sin gramatica.

Etapa final

- Ejecutar kernels en GPU.
- Comparar contra PyTorch.
- Medir compilacion y correccion.
- Generar tablas y graficas.
- Escribir paper final.

23. Resumen en una frase

El proyecto busca demostrar que una gramatica formal puede guiar a un LLM para generar kernels GPU mas correctos que con generacion libre.

La meta no es que la IA sea perfecta. La meta es controlar y verificar lo que genera.